EthioClaim

Policy-First Motor Insurance Claim Intake
and Evidence Review Platform

Project Documentation

Product Design, Frontend, Backend, and AI Evidence Analysis

Project Team

Samuel Nigussie | Mussie Ferde | Maraky Ayalneh

INSA Summer Camp

October 24, 2026

Abstract

EthioClaim is a prototype platform that helps an insurer or claims team register motor insurance claims, link them to existing policy and vehicle records, collect vehicle-damage photographs, surface duplicate or visually similar evidence, and record a human examiner's assessment. The system uses a Next.js frontend, a NestJS core API with PostgreSQL and Prisma, and a private FastAPI service for image analysis. The AI component contains a fine-tuned YOLO11n damage detector and a DINOv2-small image-similarity search. On 374 held-out CarDD test images, the detector reaches a mAP@50 of 0.701 and a mAP@50–95 of 0.550. On 1,200 synthetic image-edit queries, the similarity search ranks the correct original first in 99.5% of cases, compared with 73.1% for perceptual hashing and 16.7% for exact hashing. All AI output is advisory: it supports a human examiner and never approves, rejects, or labels a claim as fraudulent. The results come from public data and controlled image edits, so they do not establish real-world fraud-detection performance.

Contents

1 Introduction **7**

1.1 Purpose . . . 7

1.2 Core principles . . . 7

1.3 Scope of the minimum viable product . . . 7

2 Users, Roles, and Permissions **8**

2.1 Roles . . . 8

2.2 Permission matrix . . . 9

2.3 Authorization implementation . . . 9

3 Insurance Policy Model and Prefilled Forms **9**

3.1 What a policy represents . . . 9

3.2 Who registers and owns policy records . . . 10

3.3 Policy-first registration flow . . . 10

3.4 Recommended policy data . . . 10

3.5 Which fields the intake user still enters . . . 11

3.6 Privacy during policy lookup . . . 11

3.7 Policy statuses and eligibility . . . 11

3.8 Admin policy screens . . . 12

4 Workflows and Claim Statuses **12**

4.1 Main claim workflow . . . 12

4.2 State-transition rules . . . 12

4.3 Evidence workflow . . . 13

5 System Architecture **13**

5.1 Overview . . . 13

5.2 NestJS modules . . . 14

5.3 Reliability of analysis calls . . . 14

6 Frontend Specification **15**

6.1 Stack and conventions . . . 15

6.2 Route map . . . 15

6.3 Login and session behavior . . . 16

6.4 Role-based dashboard . . . 16

6.5 New Claim wizard . . . 16

6.6 Claims list and claim detail . . . 16

6.7 Evidence gallery and analysis display . . . 17
6.8 Examiner review and assessment . . . 17
6.9 Admin screens . . . 17
6.10 States every page must handle . . . 17

7 Database Model **17**
7.1 Core entities . . . 18
7.2 Database rules . . . 19
7.3 Entity relationships . . . 19

8 API Specification **19**
8.1 Authentication and users . . . 19
8.2 Policies and reference data . . . 20
8.3 Claims . . . 20
8.4 Evidence and analysis . . . 21
8.5 Assessment and audit . . . 21
8.6 Example contracts . . . 22
8.7 Error contract . . . 22

9 Entity–Relationship Diagram **23**

10 AI Evidence Analysis **23**
10.1 Role and boundaries . . . 23
10.2 Vehicle damage detection . . . 24
10.2.1 Dataset . . . 24
10.2.2 Model and training . . . 24
10.2.3 Metrics . . . 25
10.2.4 Results . . . 25
10.2.5 Interpretation . . . 25
10.2.6 Limitations . . . 27
10.3 Image similarity retrieval . . . 27
10.3.1 Objective and methods . . . 27
10.3.2 Evaluation design . . . 27
10.3.3 Results . . . 27
10.3.4 Failure analysis . . . 28
10.3.5 Thresholds from a no-match test . . . 30
10.3.6 Limitations . . . 30
10.4 FastAPI AI service . . . 30

10.4.1 Responsibilities and ownership . . . 30
10.4.2 Endpoints . . . 31
10.4.3 Request and response . . . 31
10.4.4 Model adapters . . . 32
10.4.5 Integrating the trained models . . . 32
10.4.6 Intake limits and errors . . . 33
10.4.7 Request flow . . . 33
10.5 Model readiness gate . . . 34

11 Security, Privacy, and Auditability 34
11.1 Authentication . . . 34
11.2 Authorization and data access . . . 34
11.3 File upload protection . . . 34
11.4 Audit events . . . 34
11.5 Privacy, retention, and configuration . . . 35

12 Validation and User Experience 35
12.1 Policy, claim, and evidence validation . . . 35
12.2 Accessibility and low-bandwidth use . . . 35

13 Testing and Acceptance Criteria 35
13.1 Role and authorization tests . . . 35
13.2 Policy, claim, and evidence tests . . . 35
13.3 Assessment tests . . . 36
13.4 AI service tests . . . 36
13.5 Evaluation metrics . . . 36
13.6 Definition of done . . . 36

14 Team Split and Implementation Plan 37
14.1 Order of work . . . 37
14.2 Integration rules . . . 38

15 Limitations, Ethics, and Future Work 38
15.1 Limitations . . . 38
15.2 Ethical use . . . 38
15.3 Future work . . . 38

16 Conclusion 38

A Demonstration Data and Test Scenarios 39

B AI Artifacts and Evaluation Outputs **39**

C Limitations and Future Considerations **39**

1 Introduction

1.1 Purpose

EthioClaim centralizes the motor-claim intake process and reduces repeated data entry. A policy record is registered before a claim is created. The policy number retrieves known policyholder and vehicle details, the intake user adds incident-specific information and photographs, and an examiner reviews the evidence and records an assessment.

This document is the shared source of truth for product behavior, roles and permissions, frontend screens, backend services, API contracts, data entities, AI evidence analysis, security, testing, and implementation order. It is a product and engineering specification, not legal or insurance advice. Policy eligibility, required documents, claim deadlines, decision authority, and retention periods must be confirmed with a participating insurer. The demonstration uses clearly labelled fictional policy data.

1.2 Core principles

- **Policy-first:** a policy is registered before a claim is created.
- **Enter only what is missing:** prefill known details and ask the user to confirm them.
- **Least privilege:** users access only the records and actions their role requires.
- **Human review:** AI output is advisory and never rejects a claim or labels a person as fraudulent.
- **Traceability:** record who performed important actions, when, and why for exceptional changes.
- **Resilient evidence handling:** a failed AI request never deletes the claim or the uploaded photo.
- **MVP realism:** use demo policy data until an authorized insurer integration exists.

1.3 Scope of the minimum viable product

Included:

- Login and role-based access for Admin, Intake User, and Examiner.
- Admin-managed demo policyholders, policies, and insured vehicles.
- Policy-number lookup and form prefill.
- Claim creation, draft editing, submission, status tracking, and a claim timeline.
- Photo upload, private storage, preview, and evidence metadata.
- SHA-256 exact duplicate checks and perceptual-hash near-duplicate checks.
- Image-embedding similarity search and damage detection, each shown only when tested and available.
- Examiner's manual severity, estimated loss in Ethiopian birr (ETB), notes, and recommendation.
- Admin oversight, examiner assignment, and a basic audit log.

Explicitly excluded:

- OCR or automatic extraction of text from documents or photographs.
- AI-generated-image detection, manipulation forensics, or authenticity guarantees.
- Automatic repair-cost or severity prediction.
- Automatic claim approval, rejection, payment, or definitive fraud detection.
- Direct integration with real insurer databases and third-party payment, garage, SMS, or identity services.
- A full production policy-administration suite.

2 Users, Roles, and Permissions

Authorization is enforced by the NestJS backend on every protected endpoint, using role checks plus record-level access checks. Hiding a button in the frontend is not a security control. The acting user is always derived from the authenticated session, and the backend never trusts a role, user identifier, policy status, or examiner assignment supplied by the browser.

2.1 Roles

Admin.

- Views aggregate counts, claim queues, failed analyses, and basic operational metrics.
- Creates accounts, assigns roles, activates or deactivates accounts, and initiates password resets.
- Creates, updates, and searches policy, policyholder, and vehicle records; prevents duplicate policy identifiers within an insurer.
- Oversees claims, assigns or reassigns examiners, and inspects status history.
- Views the audit log, which can never be edited or erased.
- Cannot read plaintext passwords, silently rewrite past assessments, or declare fraud automatically.

Intake User.

- Verifies a policy number and sees the minimum details needed to register a claim.
- Creates claims, saves and edits drafts, confirms prefilled data, enters incident details, uploads photographs, and submits claims.
- Views claims they created, tracks status, and responds to examiner requests.
- Sees only a limited AI summary and evidence warnings, and cannot alter original AI results.
- Cannot edit policy records, assign examiners, record assessments, or approve or reject claims.

Examiner.

- Views assigned claims with policy, vehicle, incident, evidence, and timeline information.
- Reviews original images, damage-detection output, exact duplicates, near-duplicate matches, and analysis status.
- Requests more information or evidence and explains what is missing.

- Records severity, affected areas, observations, a manual estimated loss in ETB, supporting reasoning, and a recommendation.
- May re-run analysis for an assigned claim if supported.
- Cannot manage users or policies, edit the model's original output, or delete audit history. Final approval or rejection is configurable and, if enabled, is a separate action with a required reason.

2.2 Permission matrix

Action	Admin	Intake User	Examiner
System-wide dashboard	Yes	No	No
Manage users and roles	Yes	No	No
Create or edit policy records	Yes	No	No
Verify policy	Yes	Yes	Read-only
Create or edit draft claim	Exceptional	Yes	No by default
Submit claim	Limited	Yes	No by default
View all claims	Operational need	No	No by default
View own or assigned claims	Yes	Yes (permitted scope)	Yes (assigned scope)
Upload evidence	Exceptional	Yes	Optional
View AI results and matches	Operational need	Limited summary	Yes (assigned claims)
Request additional evidence	Optional	Respond	Yes
Record severity and loss assessment	No by default	No	Yes
Recommend outcome	No by default	No	Yes
Assign examiner	Yes	No	No
View audit log	Yes	Own or limited history	Claim and own activity
Modify audit log	Never	Never	Never

2.3 Authorization implementation

The backend uses an authentication guard, a roles guard with a decorator, and service-level ownership and assignment checks. The role enumeration is `ADMIN`, `INTAKE_USER`, and `EXAMINER`. Record access: an Admin can access records for operational management, an Intake User can access claims they created or were assigned to handle, and an Examiner can access claims assigned to them. Scopes are widened only through explicit permissions.

3 Insurance Policy Model and Prefilled Forms

3.1 What a policy represents

An insurance policy is an agreement between an insurer and a policyholder that records the cover, the insured subject (here a vehicle), the effective dates, the status, and the terms. A policy number

identifies a policy record within the issuing insurer's system and is not a unique identifier for a person. One person may hold several policies, a policy may cover several vehicles, and the driver at the time of an incident may differ from the policyholder. Policyholder, insured vehicle, claimant, and driver are therefore separate concepts.

3.2 Who registers and owns policy records

In the MVP, an Admin registers fictional test policy records inside EthioClaim. In a real deployment, policy data would come from the insurer's authoritative system through an authorized interface or controlled import. A demo policy must never be presented as a real insurance contract.

3.3 Policy-first registration flow

1. The Admin registers a policyholder, a policy, and an insured vehicle.
2. The Intake User opens *New Claim* and enters the policy number.
3. The frontend sends the number to NestJS, which validates it and searches the policy database.
4. The backend returns a minimal policy summary only if the caller is authorized.
5. The frontend prefills policyholder and vehicle details and marks them as retrieved from the policy record.
6. The Intake User confirms the vehicle and policy and enters incident date and time, location, description, driver details if needed, and photographs.
7. At submission, the backend rechecks policy status and dates and does not rely on the earlier lookup.
8. The claim is created with an internal policy identifier as foreign key; the policy number is used for lookup only.
9. A snapshot of the policy and vehicle details is stored, or a versioned history is kept, so later policy edits do not silently rewrite historical context.

3.4 Recommended policy data

Entity	**Recommended fields**	**Purpose**
Insurer	id, name, code, status	Supports several insurers; policy numbers may overlap.
Policyholder	id, type (person or organization), full or legal name, phone, email, address	Identifies who is named on the policy; collect only necessary data.
Policy	id, insurer, policyholder, policy number, type, start and end date, status	Core coverage record and eligibility lookup.
Insured vehicle	id, plate number, make, model, model year, vehicle type, VIN or chassis number	Prefills vehicle details and ties claims to the correct vehicle.
Policy coverage	coverage type, deductible, limits, exclusions	Useful for real assessments; optional in a short MVP.
Audit event	actor, action, entity, timestamp, reason	Tracks policy creation and changes.

The MVP may support one policyholder and one vehicle per policy, but the schema does not assume that a policy number identifies a person. If several vehicles per policy are in scope, a `PolicyVehicle` join table is used. When several insurers are supported, uniqueness is enforced on the pair of insurer and normalized policy number, not on the policy number alone.

3.5 Which fields the intake user still enters

Field	Source	Intake action
Policy number	User-provided lookup key	Enter it; do not re-enter the full policy.
Policyholder name	Policy record	Prefill and confirm; avoid arbitrary edits.
Policy status and dates	Policy record	Display read-only with a clear eligibility warning.
Vehicle plate, make, model	Insured-vehicle record	Prefill and confirm the vehicle involved.
Incident date and time	Claimant report	Required.
Incident location	Claimant report	Required if the insurer's workflow needs it.
Incident description	Claimant report	Required short description.
Driver identity	Incident-specific	Collect only when needed.
Claimant contact details	Policyholder record or claimant	Prefill only if appropriate; update through a controlled workflow.
Damage photographs	Intake user	Upload; each image becomes an evidence record.
Relationship to policyholder	Claimant	Optional or required per the insurer's process.

3.6 Privacy during policy lookup

A policy number alone is not proof of identity. The lookup returns only the fields needed to register the claim. In production, appropriate authentication or an insurer-approved identity check is required before personal data is revealed, and phone numbers should be masked.

3.7 Policy statuses and eligibility

Status	Suggested behavior
`ACTIVE`	Allow claim registration, subject to policy dates and insurer rules.
`EXPIRED`	Show a warning and block or route registration per configured rules; the intake user cannot override it.
`CANCELLED`	Block normal registration or route to an authorized exception process.
`SUSPENDED`	Block or flag for admin review.
`PENDING`	Do not treat as active coverage unless the insurer's rules explicitly permit it.
`NOT_FOUND`	Offer correction and retry with a controlled escalation path; never create a policy from the intake form.

A valid policy lookup does not establish that a specific incident is covered; coverage depends on the policy terms and insurer review.

3.8 Admin policy screens

The admin interface provides a policy list (search by policy number, policyholder, plate, insurer, and status, with pagination), a creation form grouped by policyholder, insurer, dates, vehicle, and coverage, a detail view with linked claims and change history, controlled edits with a required reason for sensitive changes, deactivation with confirmation instead of hard deletion, and a clear message when a duplicate policy number is entered.

4 Workflows and Claim Statuses

4.1 Main claim workflow

1. `DRAFT`: the Intake User has started a claim and can edit it.
2. `SUBMITTED`: the claim is validated and receives its official reference number.
3. `UNDER_REVIEW`: the claim is assigned to an examiner and available for review.
4. `INFO_REQUESTED`: the examiner asked for more details or evidence.
5. `INFO_RECEIVED`: the Intake User supplied the information and the claim returns to the examiner's queue.
6. `ASSESSMENT_SUBMITTED`: the examiner recorded the assessment and recommendation.
7. `APPROVED` or `REJECTED`: used only if a final-decision capability is included.
8. `CLOSED`: the workflow is complete; reopening is controlled and logged.

Claim status is kept separate from evidence-analysis status. A claim can be `UNDER_REVIEW` while one image's analysis has failed, and this does not put the whole claim into an error state. Analysis states such as pending, completed, partial, failed, or unavailable belong to the evidence-analysis record only.

4.2 State-transition rules

From	To	Initiated by	Rule
`DRAFT`	`SUBMITTED`	Intake User	Required fields and policy checks pass.
`SUBMITTED`	`UNDER_REVIEW`	System or Admin	Claim is queued and assigned or available for assignment.
`UNDER_REVIEW`	`INFO_REQUESTED`	Examiner	Reason and request details required.
`INFO_REQUESTED`	`INFO_RECEIVED`	Intake User or system	Requested response or evidence is recorded.
`INFO_RECEIVED`	`UNDER_REVIEW`	System or Examiner	Returns to the examiner queue.

From	To	Initiated by	Rule
UNDER_REVIEW	ASSESSMENT_SUBMITTED	Examiner	Severity, reasoning, and recommendation validated.
ASSESSMENT_SUBMITTED	APPROVED / REJECTED	Authorized decision-maker	Separate permission, required reason, audit event.
Any allowed state	CLOSED	Authorized role	Closure rules; record the reason.
CLOSED	Reopened state	Admin or authorized role	Explicit reason and audit event.

4.3 Evidence workflow

1. The upload request is authorized against the claim and validated for file size and type.
2. The file is stored in private storage and its metadata is written to the database.
3. The exact hash (SHA-256) is calculated; an identical hash marks the file as an exact-duplicate candidate.
4. The perceptual hash flags likely near-duplicates within a tested threshold.
5. Embedding similarity is shown as a similarity candidate, not as proof of reuse or fraud.
6. Damage inference runs only when the model is available and tested.
7. Analysis output and status are stored separately from the original evidence record.
8. The examiner reviews the findings and records a human conclusion. A failed analysis can be retried without losing the evidence.

5 System Architecture

5.1 Overview

The browser calls only the NestJS application API. NestJS owns authentication, authorization, persistence, the claim lifecycle, and the official API contract. The FastAPI service performs image analysis and returns structured technical observations; it is private and is never exposed to the browser.

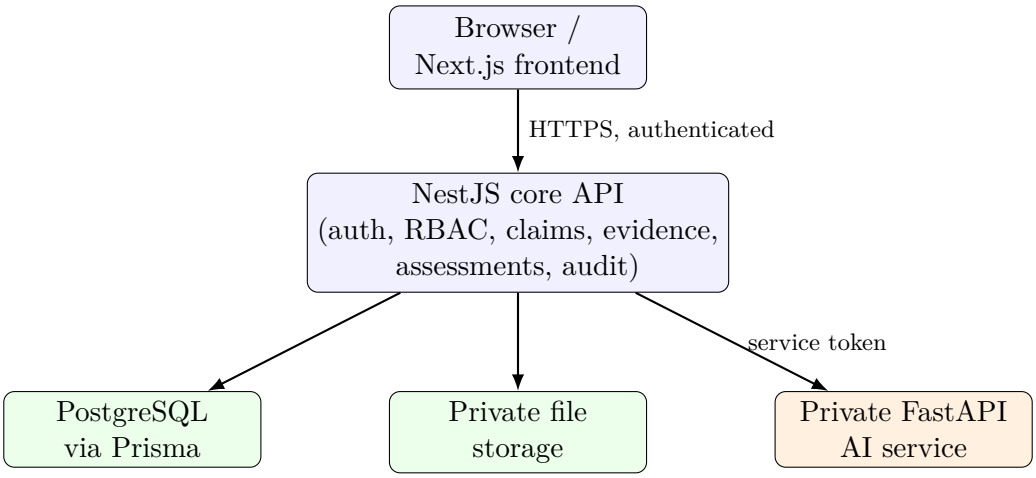


Figure 1: High-level architecture. Only NestJS is reachable from the browser.

5.2 NestJS modules

Module	Responsibilities
`AuthModule`	Login, logout, current user, password hashing, session or token lifecycle.
`UsersModule`	Admin-only user management and role assignment.
`InsurersModule`	Insurer records when several insurers are supported.
`PolicyholdersModule`	Policyholder records and minimal profile data.
`PoliciesModule`	Policy CRUD, lookup, status and date checks, duplicate prevention.
`VehiclesModule`	Vehicle records and association to policies.
`ClaimsModule`	Claim creation, list and detail, status transitions, assignment, timeline.
`EvidenceModule`	Upload authorization, metadata, file access, deletion and replacement policy.
`AnalysisModule`	AI-service calls, result persistence, retry state, duplicate matches.
`AssessmentsModule`	Severity, estimated loss, examiner notes, recommendation or decision.
`AuditModule`	Append-only event creation and authorized audit queries.
`HealthModule`	Health and readiness checks without revealing secrets.

5.3 Reliability of analysis calls

For a compact MVP, NestJS creates the evidence record first and then calls FastAPI synchronously. The upload response must not fail merely because AI inference fails. NestJS sets connection and read timeouts, stores an analysis status and an error category (no stack traces in user-visible responses), uses the evidence identifier or an idempotency key to avoid duplicate jobs on retries, permits retries only for eligible failed or pending analyses while keeping earlier analysis versions, and never blocks examiner review because a model is unavailable. A job queue (for example Redis with BullMQ) is optional and can be added if measured latency requires it.

6 Frontend Specification

6.1 Stack and conventions

- Next.js App Router with TypeScript and a shared authenticated layout with navigation filtered by role.
- A single typed API client for the NestJS endpoints; the frontend never calls FastAPI directly.
- Forms with client-side feedback plus server-side validation messages.
- Reusable components for tables, status badges, image upload and preview, confirmation dialogs, and empty and error states.
- Server-side pagination, search, and filtering for lists.
- Accessible labels, keyboard navigation, visible focus states, responsive layouts, and readable contrast.

6.2 Route map

Route	Who	Purpose
`/login`	Unauthenticated users	Login and validation errors.
`/dashboard`	All authenticated roles	Role-specific summary and actions.
`/claims`	Admin; authorized Intake and Examiner	Search and filter claims within access scope.
`/claims/new`	Intake User	Policy-first claim registration wizard.
`/claims/[claimId]`	Authorized roles	Claim overview, details, status, timeline.
`/claims/[claimId]/evidence`	Authorized roles	Evidence gallery and analysis status.
`/claims/[claimId]/review`	Assigned Examiner	Review images, matches, and analysis.
`/claims/[claimId]/assessment`	Assigned Examiner	Severity, loss estimate, findings, recommendation.
`/policies`	Admin	Policy search and list.
`/policies/new`	Admin	Register policyholder, policy, and vehicle.
`/policies/[policyId]`	Admin; limited read-only for others	Policy detail and linked claims.
`/admin/users`	Admin	Manage users and roles.
`/admin/audit-log`	Admin	Search important system events.
`/admin/operations`	Admin	Analysis failures and operational monitoring.

6.3 Login and session behavior

The login form takes an email or username and a password. Invalid credentials produce a generic error that does not reveal whether an account exists. After success, the frontend fetches the current user and role and routes to the dashboard. Expired sessions and logout are handled, secrets are not kept in insecure browser storage, unauthenticated users are redirected to login, and unauthorized users see a safe forbidden page.

6.4 Role-based dashboard

Role	Dashboard cards and actions
Admin	Total claims, unassigned claims, claims by status, active and expired policy counts, failed analysis jobs, and links to users, policies, claim queue, and audit log.
Intake User	Drafts, submitted claims, claims needing more information, recent claims, and a New Claim action.
Examiner	Assigned queue, claims awaiting review, information requested, assessments in progress, and cases whose analysis failed.

Dashboard metrics come from authorized backend responses; privileged counts are never derived by downloading all claims into the browser.

6.5 New Claim wizard

1. **Find policy:** enter the policy number and click Verify.
2. **Confirm policy and vehicle:** show status, dates, policyholder name, and vehicle details as read-only values from the policy record, with clear warnings for not found, expired, cancelled, or inactive policies.
3. **Incident details:** date and time, location, narrative, driver details and claimant relationship if required.
4. **Evidence:** upload several photographs, preview and remove them before submission, add optional descriptions, and show upload progress.
5. **Review and submit:** show a summary, require confirmation that the information is accurate, and submit.
6. **After submission:** show the claim reference number, the submitted status, and the next steps.

Policyholder name, policy dates, make, model, and plate are not retyped when reliably stored. A correction-request path replaces silent edits of authoritative policy data.

6.6 Claims list and claim detail

The claims list offers server-side pagination, keyword search, status filter, date range, and role-appropriate visibility. Columns are claim reference, masked policy number, vehicle plate as permitted, submitted date, status, assigned examiner where appropriate, and latest update. The claim detail has the sections Overview, Policy and Vehicle, Incident, Evidence, Analysis, Assessment, and Timeline, shows the status history with actor and time, shows actions only when the role and current status allow them, and uses confirmation dialogs for submission, reassignment, and closure.

6.7 Evidence gallery and analysis display

- A grid of images with thumbnail, upload time, uploader, description, and processing status, with a full-size preview.
- Original evidence is shown separately from analysis results so model output cannot be mistaken for the original image.
- Exact duplicates, near-duplicates, and embedding-similarity candidates are visually distinct.
- A related claim or evidence reference is shown only if the current user is authorized to view it.
- AI output is labelled advisory, and confidence is shown only when returned by the real model.
- If analysis fails, the failure status is shown, the image is retained, manual review stays available, and a retry action appears only for permitted roles.

Recommended result labels: "Exact match found", "Possible visual match", "No match found", "Analysis incomplete", and "Analysis unavailable". A similarity score is displayed with a note that it is not a fraud probability.

6.8 Examiner review and assessment

The review screen shows the claim summary, policy and vehicle data, an image viewer, the AI summary, duplicate matches, and notes together. The examiner selects a severity from insurer-approved options (initially Minor, Moderate, Severe, and Unable to Determine), enters a manual estimated loss in ETB with supporting notes, and records observations, concerns, and a recommendation. Requesting information requires a clear message and sets the claim to `INFO_REQUESTED`. A review summary and a confirmation that the assessment is the examiner's own judgement precede submission. If a final decision is implemented, it is separate from the recommendation and requires a reason.

6.9 Admin screens

The admin screens include a policy list with filters and pagination (no hard-delete control for policies referenced by claims), grouped policy creation forms, policy detail with linked claims and change history, a user list showing name, login, role, status, and last login (never password hashes), confirmed and logged role changes and deactivations, and an audit log filterable by actor, action, entity, and date.

6.10 States every page must handle

Loading, upload progress, and analysis processing; empty, not-found, and forbidden states; validation, server, network-timeout, and session-expiry errors; duplicate policy number and policy not found; inactive or expired policy warnings; file too large, unsupported type, and interrupted upload; no AI model configured or analysis unavailable; and responsive layouts for low-end devices and narrow screens.

7 Database Model

PostgreSQL is accessed through Prisma. Identifiers are opaque internal IDs (UUID or CUID) with foreign keys and indexes. The schema below is logical and is adapted to the chosen Prisma version.

7.1 Core entities

Entity	Key fields	Notes
User	id, displayName, email or username, passwordHash, role, isActive, createdAt, updatedAt	Unique login identifier; only the password hash is stored.
Insurer	id, name, code, status	Recommended for policy-number scoping.
Policyholder	id, entityType, full or legal name, phone, email, address, createdAt	Separate from user accounts.
Vehicle	id, plateNumber, make, model, modelYear, vehicleType, VIN or chassis number	Avoid unnecessary identifiers.
Policy	id, insurerId, policyholderId, policyNumber, status, startDate, endDate, policyType	Unique per insurer and normalized policy number.
PolicyVehicle	policyId, vehicleId	For several vehicles per policy.
Claim	id, claimNumber, policyId, vehicleId, createdById, assignedExaminerId, status, incidentAt, location, description, submittedAt	The claim number is human-friendly; the id is the internal key.
ClaimPolicySnapshot	claimId, snapshotJson, capturedAt	Preserves policy and vehicle context at submission.
Evidence	id, claimId, uploadedById, storageKey, originalFilename, mimeType, sizeBytes, sha256, perceptualHash, description, uploadedAt, status	The file lives in private storage; the database stores metadata and the key.
AnalysisRun	id, evidenceId, status, modelName, modelVersion, damageOutputJson, sha256, perceptualHash, embeddingRef, errorCode, startedAt, completedAt	One evidence item can have several runs.
EvidenceMatch	id, evidenceId, matchedEvidenceId, method, score, createdAt	Method is SHA256, PHASH, or EMBEDDING.
Assessment	id, claimId, examinerId, severity, estimatedLossETB, recommendation, notes, createdAt, updatedAt, submittedAt	Keep history if submitted versions can change.
ClaimEvent	id, claimId, actorUserId, eventType, fromStatus, toStatus, message, metadataJson, createdAt	Timeline and history.
AuditLog	id, actorUserId, action, entityType, entityId, timestamp, reason, metadataJson	Append-only from the application's perspective.

7.2 Database rules

- Unique login identifier for users, and unique (insurer, normalized policy number) for policies.
- A claim references an existing policy and, optionally, the selected insured vehicle; the vehicle must belong to the policy.
- Evidence references a claim and an uploader; file metadata never replaces authorization checks.
- Monetary estimates use `Decimal/NUMERIC`, never floating point, and the currency (ETB) is named in the API and interface.
- Transactions cover operations that create a claim with its timeline event, or submit an assessment and change status.
- Policies and users are deactivated, not hard-deleted, and nothing cascades in a way that destroys claim history.
- Indexes cover claim number, policy number, plate number, status, assigned examiner, creation date, evidence hash, and foreign keys.
- A JSON field is used only for flexible analysis metadata; fields used for filtering stay structured.

7.3 Entity relationships

```
Insurer 1 ---- * Policy * ---- 1 Policyholder
                 |
                 * (PolicyVehicle if several vehicles)
                 |
               Vehicle
                 |
               * Claim ---- * Evidence ---- * AnalysisRun
                 | |
                 | * EvidenceMatch * ---- Evidence
                 |
                 * Assessment
                 |
                 * ClaimEvent

User creates Claims, uploads Evidence, performs Assessments
User appears as actor in AuditLog
```

8 API Specification

The base URL is `/api/v1`. Field names and response DTOs are shared between frontend and backend developers, errors have a consistent shape, and HTTP status codes are used appropriately. An OpenAPI specification should be committed early.

8.1 Authentication and users

Method and path	Access	Purpose
`POST /auth/login`	Public	Authenticate; return a session or token.
`POST /auth/logout`	Authenticated	Invalidate the session.

Method and path	Access	Purpose
`GET /auth/me`	Authenticated	Current user id, name, role, and status.
`GET /admin/users`	Admin	Paginated user list.
`POST /admin/users`	Admin	Create a user with an allowed role.
`GET /admin/users/:userId`	Admin	User detail without password hash.
`PATCH /admin/users/:userId`	Admin	Update permitted fields or status.
`POST /admin/users/:userId/reset-password`	Admin	Initiate a reset; never return a password.

8.2 Policies and reference data

Method and path	Access	Purpose
`POST /policies/verify`	Authenticated	Look up a policy number; return found, status, and a minimal summary.
`GET /admin/policies`	Admin	Search, filter, and paginate.
`POST /admin/policies`	Admin	Create a policy and its associations.
`GET /admin/policies/:policyId`	Admin	Policy detail.
`PATCH /admin/policies/:policyId`	Admin	Controlled corrections with an audit reason.
`GET, POST /admin/policyholders`	Admin	Search or create policyholders.
`GET, POST /admin/vehicles`	Admin	Search or create vehicles.

8.3 Claims

Method and path	Access	Purpose
`POST /claims`	Intake User	Create a draft claim linked to a policy.
`GET /claims`	Role-scoped	List with server-side filters and pagination.
`GET /claims/:claimId`	Record-scoped	Claim details if the caller has access.
`PATCH /claims/:claimId`	Intake User (draft)	Edit allowed fields.
`POST /claims/:claimId/submit`	Intake User	Revalidate policy, fields, and evidence, then submit.
`PATCH /claims/:claimId/assign`	Admin	Assign or reassign an examiner.
`PATCH /claims/:claimId/status`	Role and status policy	Allowed transitions only.

Method and path	Access	Purpose
`GET /claims/:claimId/timeline`	Record-scoped	Ordered claim events.
`POST /claims/:claimId/request-information`	Assigned Examiner	Request missing information with a message.
`POST /claims/:claimId/close`	Authorized role	Close with a reason.

8.4 Evidence and analysis

Method and path	Access	Purpose
`POST /claims/:claimId/evidence`	Intake User	Upload a file and create evidence metadata.
`GET /claims/:claimId/evidence`	Record-scoped	List evidence metadata.
`GET /evidence/:evidenceId`	Record-scoped	Metadata and permitted analysis summary.
`GET /evidence/:evidenceId/file`	Record-scoped	Authorize, then stream the file or issue a short-lived signed URL.
`GET /evidence/:evidenceId/analysis`	Record-scoped	Latest analysis status and results.
`GET /evidence/:evidenceId/matches`	Examiner, Admin	Duplicate and similarity candidates with access-controlled details.
`POST /evidence/:evidenceId/reanalyze`	Assigned Examiner, Admin	Queue or retry analysis.
`DELETE /evidence/:evidenceId`	Restricted	Only before submission or through an audited process.

8.5 Assessment and audit

Method and path	Access	Purpose
`POST /claims/:claimId/assessment`	Assigned Examiner	Create or submit the assessment.
`GET /claims/:claimId/assessment`	Record-scoped	Current assessment and permitted history.
`PATCH /claims/:claimId/assessment`	Assigned Examiner	Update a draft assessment; preserve history.
`POST /claims/:claimId/decision`	Authorized (optional)	Record a final decision with a required reason.
`GET /admin/audit-log`	Admin	Search audit events.
`GET /admin/operations/analysis`	Admin	Pending and failed analysis jobs.
`GET /health`	Public or internal	Minimal health check.

8.6 Example contracts

The values below are fictional illustrations.

```
POST /api/v1/policies/verify
Request: { "policyNumber": "MOT-2026-00125" }
Response: {
  "found": true,
  "policy": {
    "id": "policy_internal_id",
    "policyNumber": "MOT-2026-00125",
    "status": "ACTIVE",
    "startDate": "2026-01-01",
    "endDate": "2026-12-31",
    "policyholder": { "displayName": "Abebe K." },
    "vehicle": { "plateNumber": "AA-3-12345", "make": "Toyota", "model": "Corolla" }
  }
}
```

```
POST /api/v1/claims
{
  "policyId": "policy_internal_id",
  "vehicleId": "vehicle_internal_id",
  "incidentAt": "2026-10-19T14:30:00+03:00",
  "incidentLocation": "Addis Ababa",
  "incidentDescription": "Reported collision involving the insured vehicle."
}
Response: { "id": "claim_internal_id", "claimNumber": "EC-2026-000123",
           "status": "DRAFT", "createdAt": "2026-10-20T08:30:00Z" }
```

```
POST /api/v1/claims/:claimId/assessment
{
  "severity": "MODERATE",
  "estimatedLossETB": "18000.00",
  "observations": "Visible damage to the front bumper; manual estimate.",
  "recommendation": "REQUEST_MORE_INFORMATION",
  "recommendationReason": "Additional view of the affected area is needed."
}
```

Money is a decimal string. Enumerations are validated on the backend. A recommendation of `REQUEST_MORE_INFORMATION` requires the request details and moves the claim through the appropriate workflow.

8.7 Error contract

```
{
  "statusCode": 400,
  "code": "POLICY_NOT_ELIGIBLE",
  "message": "This policy cannot be used to submit a claim through the standard workflow.",
  "details": { "policyStatus": "EXPIRED" },
  "requestId": "req_example"
}
```

Error categories: `VALIDATION_ERROR`, `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `POLICY_NOT_FOUND`, `POLICY_NOT_ELIGIBLE`, `DUPLICATE_POLICY_NUMBER`, `INVALID_STATUS_TRANSITION`, `FILE_TOO_LARGE`, `UNSUPPORTED_FILE_TYPE`, `ANALYSIS_UNAVAILABLE`, `RATE_LIMITED`, and `INTERNAL_ERROR`. Stack traces, private file paths, secrets, and database internals are never sent to the frontend.

9 Entity–Relationship Diagram

The diagram below is based on the project's current `schema.prisma` file. It shows the database entities and the relationships defined in the current Prisma schema, rather than every optional entity proposed in the conceptual design above. In particular, user-upload and decision-actor identifiers are shown as scalar UUID fields where the current schema does not define a Prisma relation.

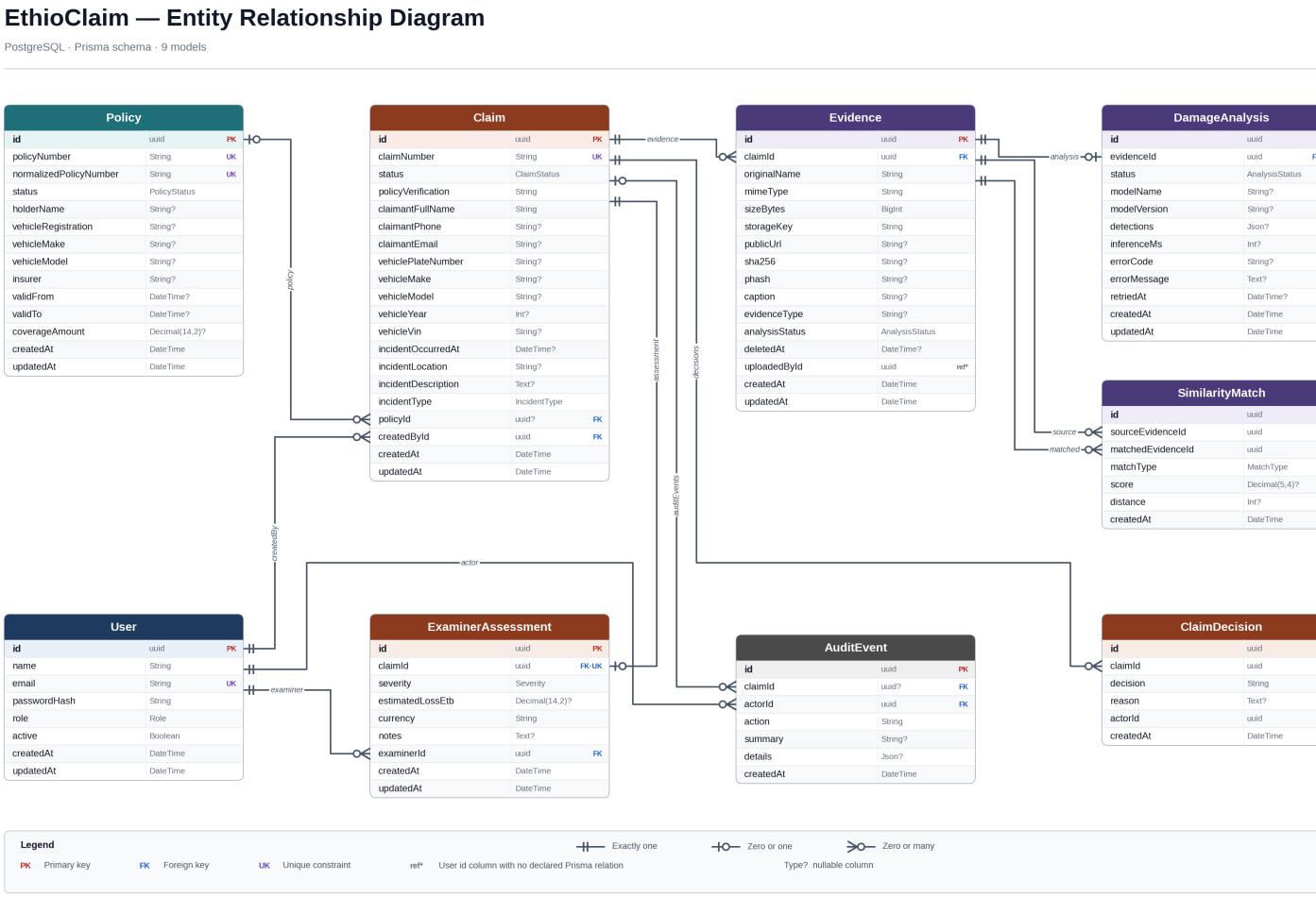


Figure 2: Entity-Relationship Diagram of the EthioClaim Database

10 AI Evidence Analysis

10.1 Role and boundaries

The AI component contains two independent parts:

1. **Vehicle damage detection:** a fine-tuned YOLO11n model locates and classifies visible damage.
2. **Image similarity retrieval:** DINOv2-small embeddings retrieve visually similar stored images to help identify possibly reused or modified photographs.

Both parts produce evidence for a human examiner. Neither declares a claim fraudulent, estimates damage severity, or estimates repair cost. A similarity score is an investigation signal and not proof

of reuse or fraud, and a detection confidence is a model score, not the probability that a claim is valid.

10.2 Vehicle damage detection

10.2.1 Dataset

The detector was trained on CarDD (Car Damage Detection), obtained from the Hugging Face copy `shanexf/cardd-damage`, a third-party conversion to Ultralytics YOLO format that keeps the official split. CarDD is distributed for non-commercial research and educational use, and the original paper is cited in the references. The six classes are dent, scratch, crack, shattered glass, broken lamp, and flat tire. The split contains 2,816 training, 810 validation, and 374 test images (4,000 in total). No label file was empty and all labels were in bounding-box format.

Class	**Train**	**Validation**	**Test**
dent	1,806	501	236
scratch	2,560	728	307
crack	651	177	70
shattered_glass	475	135	71
broken_lamp	494	141	69
flat_tire	225	62	32
Total instances	6,211	1,744	785

Table 15: Annotated instances per class and split.

The classes are imbalanced: scratch has about eleven times more training instances than flat tire. The test classes flat tire (32 instances) and crack (70) are small, so their scores are less stable than those of the larger classes.

10.2.2 Model and training

Item	**Setting**
Model	YOLO11n (Ultralytics 8.4.183), pretrained weights, head changed to six classes
Parameters	about 2.6 million (about 5.5 MB checkpoint)
Input size	640×640
Epochs and batch size	50 epochs (all completed), batch size 16
Early stopping	patience 15 epochs (not triggered)
Optimizer	AdamW selected automatically (learning rate 0.001)
Precision and augmentation	Automatic mixed precision; Ultralytics default augmentation
Seed	42
Hardware and software	NVIDIA Tesla T4 on Google Colab; Python 3.13; PyTorch 2.11
Training time	about 0.995 hours

Table 16: Training configuration.

Protocol. The validation set monitored training and selected the best checkpoint. The test set was evaluated once, after training and checkpoint selection, and nothing was changed afterwards.

10.2.3 Metrics

Precision is the share of predicted detections that are correct. *Recall* is the share of annotated damage instances the model finds. *mAP@50* is the average precision across classes when a detection counts as correct at an intersection-over-union (IoU) of at least 0.50. *mAP@50–95* averages over IoU thresholds from 0.50 to 0.95 and is stricter.

10.2.4 Results

Metric	Validation (810 images)	Test (374 images)
Precision	0.731	0.738
Recall	0.679	0.653
mAP@50	0.707	**0.701**
mAP@50–95	0.563	**0.550**

Table 17: Overall damage-detection performance.

Class	Images	Inst.	P	R	mAP@50	mAP@50–95
dent	157	236	0.680	0.547	0.602	0.339
scratch	183	307	0.627	0.518	0.558	0.306
crack	48	70	0.502	0.343	0.350	0.183
shattered_glass	71	71	0.947	0.972	0.990	0.916
broken_lamp	65	69	0.755	0.725	0.823	0.694
flat_tire	31	32	0.915	0.812	0.881	0.861

Table 18: Per-class test performance. Image counts are per class; one image can contain several classes.

Validation mAP@50 / mAP@50–95 per class was: dent 0.596 / 0.335, scratch 0.534 / 0.302, crack 0.292 / 0.156, shattered glass 0.994 / 0.948, broken lamp 0.874 / 0.740, and flat tire 0.955 / 0.899. On the T4 GPU, inference took about 2.0 ms preprocessing, 4.6 ms model, and 2.0 ms postprocessing per image.

Validation mAP@50 rose from 0.424 after the first epoch to about 0.70 by epoch 43 and then stayed nearly flat. Validation and test mAP@50–95 are close (0.563 and 0.550), so there is no clear sign of overfitting to the validation set.

10.2.5 Interpretation

The model is strong on visually distinctive damage: shattered glass (mAP@50 0.990), flat tire (0.881), and broken lamp (0.823). It is much weaker on fine surface damage: dent (0.602), scratch (0.558), and especially crack (0.350), where recall of 0.343 means roughly two thirds of annotated cracks are missed. Plausible explanations are that these damage types are thin, small, or subtle at 640 pixels, resemble one another, and, for crack, have fewer training examples. These explanations were not tested.

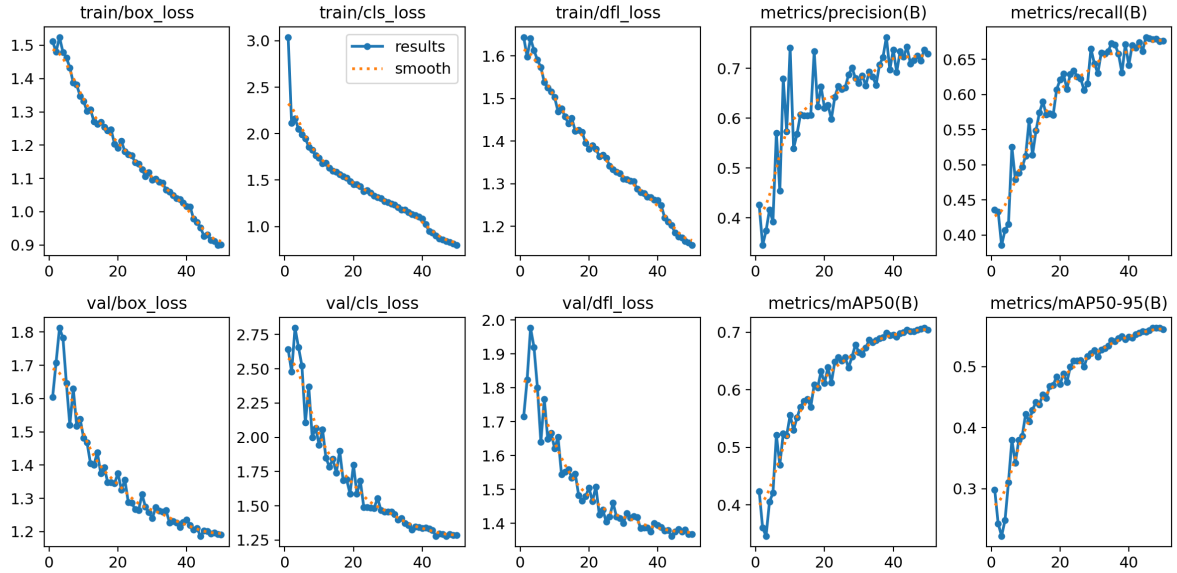


Figure 3: Training and validation curves of the YOLO11n model.

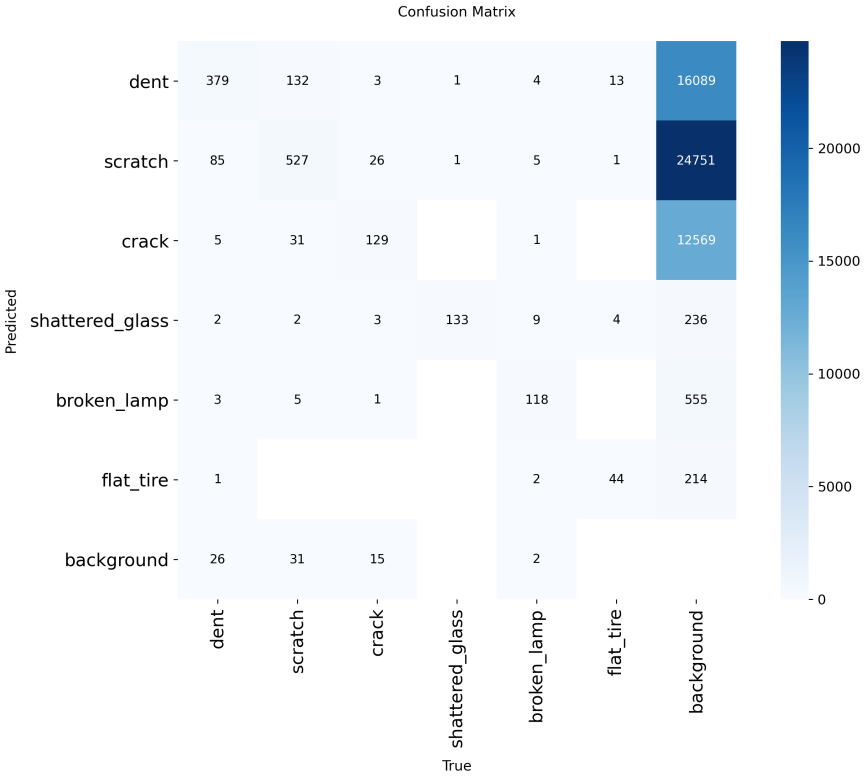


Figure 4: Confusion matrix of the YOLO11n damage detector.

10.2.6 Limitations

- A single training run with the smallest model size and no hyperparameter tuning.
- CarDD does not represent Ethiopian vehicles, roads, camera quality, or local claim photographs, so local performance is unknown.
- Lighting, occlusion, distance, and resolution may reduce accuracy.
- The model detects visible damage only; it does not assess severity, repair cost, or whether a claim is genuine.
- The packaged confidence cutoff of 0.25 is the Ultralytics default and was not tuned.
- Inference speed was measured on a GPU only.

10.3 Image similarity retrieval

10.3.1 Objective and methods

Given a new photograph, the system retrieves the most similar images from a gallery of stored photographs, giving an examiner a lead on photographs that may have been reused, resized, recompressed, cropped, or otherwise modified. Three methods were compared:

1. **SHA-256:** a cryptographic hash of the file bytes; it matches only byte-identical files.
2. **Perceptual hash (pHash):** a compact 64-bit fingerprint of image content, compared by Hamming distance.
3. **DINOv2-small embeddings:** a pretrained vision model (`facebook/dinov2-small`) converts each image to a 384-dimensional vector (the CLS-token output, L2-normalized), and images are ranked by cosine similarity. The model was not trained or fine-tuned in this project.

10.3.2 Evaluation design

CarDD has no labels for duplicate or related images, so the ground truth was constructed. The gallery was the 374 CarDD test images. Two hundred gallery images were selected at random (seed 42) and each was turned into six queries, giving 1,200 queries: an exact copy, a resize to 50%, a re-save as JPEG at quality 30, a central 80% crop, a 40% brightness increase, and a horizontal flip. A query is correct if its source image is ranked first (Recall@1) or among the top five (Recall@5).

10.3.3 Results

Method	Recall@1	Recall@5
SHA-256	0.167	0.167
pHash	0.731	0.771
DINOv2-small	**0.995**	**0.998**

Table 19: Overall retrieval performance on 1,200 synthetic queries.

Method	Exact	Resize	JPEG	Crop	Bright.	Flip	Overall
SHA-256	1.000	0.000	0.000	0.000	0.000	0.000	0.167
pHash	1.000	1.000	1.000	0.385	1.000	0.000	0.731
DINOv2-small	1.000	1.000	1.000	0.970	1.000	1.000	0.995

Table 20: Recall@1 by transformation.

Method	Exact	Resize	JPEG	Crop	Bright.	Flip	Overall
SHA-256	1.000	0.000	0.000	0.000	0.000	0.000	0.167
pHash	1.000	1.000	1.000	0.625	1.000	0.000	0.771
DINOv2-small	1.000	1.000	1.000	0.990	1.000	1.000	0.998

Table 21: Recall@5 by transformation.

SHA-256 scores exactly one sixth because only one of the six query types keeps the file bytes unchanged. pHash handles resizing, recompression, and brightness perfectly but fails on flips and is weak on crops. DINOv2-small ranked the correct original first for 1,194 of 1,200 queries.

10.3.4 Failure analysis

All six DINOv2 top-1 failures were cropped queries. In the two inspected visually, the system returned a different vehicle with similar damage (similarity 0.59 for a bumper scratch and 0.74 for a shattered windshield) instead of the true original. The embedding therefore captures the type of damage as well as the individual photograph, so unrelated claims with similar damage can score as similar.

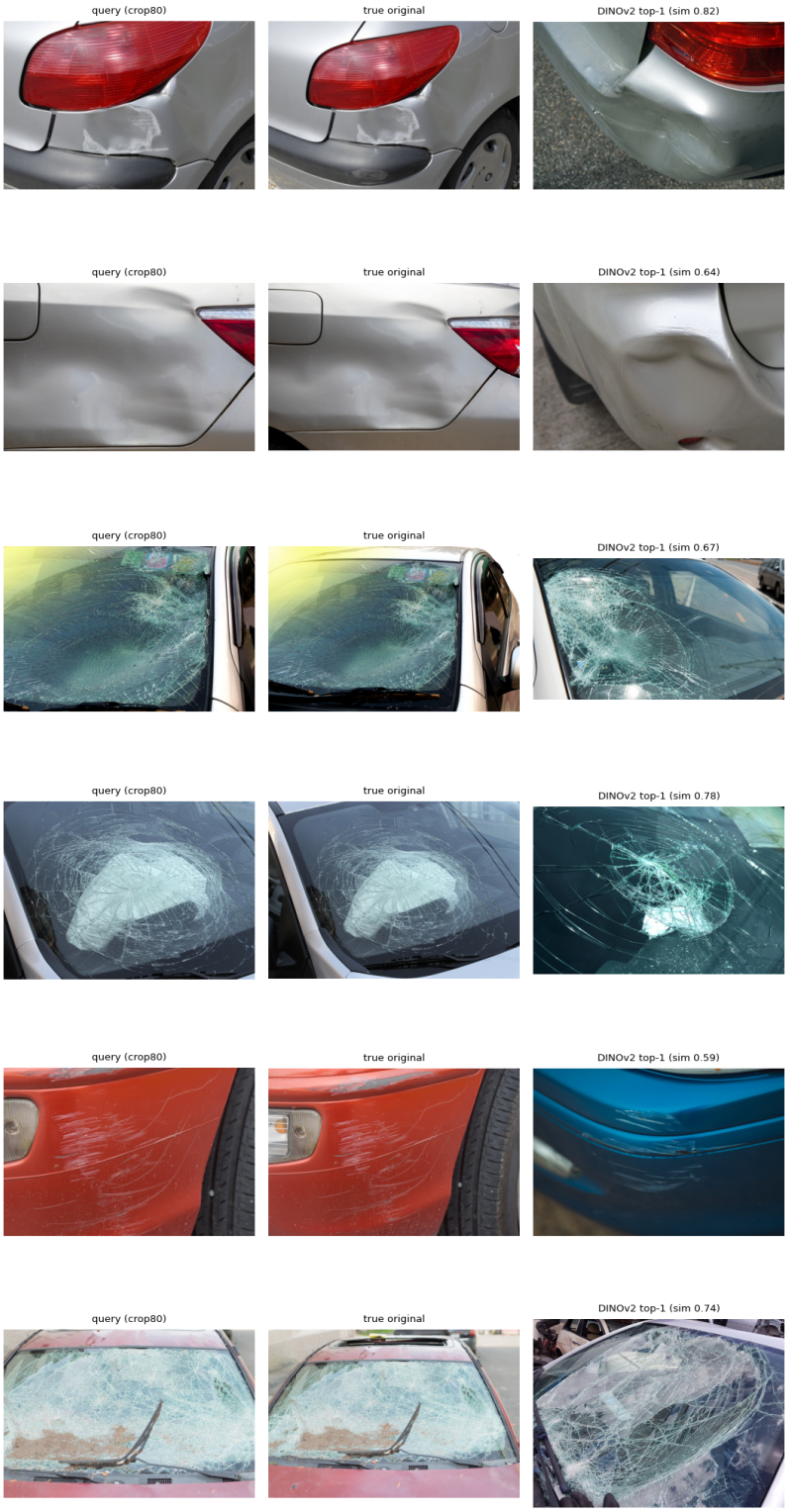


Figure 5: Examples of DINOv2 image-similarity failures. Each example shows the cropped query image, the correct original image, and the incorrectly retrieved match.

10.3.5 Thresholds from a no-match test

To see how scores behave when no true match exists, 100 of the 200 source images were removed from the gallery, leaving 274 images. Their 600 queries (100 sources × 6 transformations) had no true match, and the other 600 queries still had theirs. A query was flagged when its top-1 cosine similarity reached a threshold. The median top-1 similarity was 0.979 for queries with a true match and 0.689 for queries without one (95th percentile 0.819).

Threshold	True matches found	False alarms (no true match)
0.5	99.8%	98.2%
0.6	99.8%	87.0%
0.7	99.2%	43.7%
0.8	97.3%	11.8%
0.9	88.3%	0.0%

Table 22: Effect of the similarity threshold.

Cosine similarity	Interpretation	Suggested action
0.90 or higher	Strong match	Flag for examiner review
0.80 to below 0.90	Similar image	Show as a candidate
below 0.80	No flag	Do not flag

Table 23: Provisional similarity policy for the prototype.

These values are starting points and not validated production settings. A score of 0.90 is not a 90% probability that two images are the same.

10.3.6 Limitations

- The gallery is small (374 images) and the no-match images are random unrelated vehicles, which is an easy case. The test contains no hard negatives such as different photographs of the same vehicle or accident, or different vehicles with similar damage and backgrounds. Real false-alarm rates are likely to be higher.
- The 600 no-match queries come from only 100 distinct images (six correlated variants each), so false-alarm rates are less precise than the number of queries suggests.
- The queries are synthetic edits of existing images, not photographs from real claims. The results show robustness to those edits and not detection of fraud.
- The perceptual-hash Hamming threshold was not calibrated; methods were compared by ranking only.
- Thresholds must be recalibrated on representative data before operational use.

10.4 FastAPI AI service

10.4.1 Responsibilities and ownership

FastAPI accepts a trusted image reference created by NestJS, validates and normalizes the image, computes SHA-256 and perceptual hashes, runs the damage detector, optionally creates embeddings

and retrieves similar evidence, and returns structured results with model versions, warnings, and explicit component status. It does not own users, policies, claim lifecycle, assessments, repair-cost estimates, or decisions, and it is called only by NestJS.

Concern	Owner	Notes
Users, roles, authorization	NestJS	All browser-facing authentication and authorization.
Policies and claim workflow	NestJS	FastAPI does not verify coverage or change claim status.
Original evidence storage	NestJS and object storage	The original is stored once; a controlled reference is passed to AI.
Image analysis	FastAPI	Preprocessing, inference, hashing, embeddings.
Human assessment and decision	NestJS and PostgreSQL	Examiner severity and cost remain independent of AI.
AI result persistence	NestJS and PostgreSQL	Results linked to evidence ID, with model versions.

10.4.2 Endpoints

Method and path	Purpose	MVP priority
`GET /health/live`	Process is alive	Required
`GET /health/ready`	Dependencies and models ready	Required
`POST /internal/v1/analyze`	Analyze one evidence image	Required
`POST /internal/v1/duplicates/check`	Exact and pHash lookup	Required if an index exists
`POST /internal/v1/similarity/search`	Top-k embedding search	Can be disabled initially
`GET /internal/v1/analysis/{id}`	Retrieve a stored result	Optional
`POST /internal/v1/analysis-jobs`	Queue an asynchronous analysis	Optional
`GET /internal/v1/analysis-jobs/{id}`	Poll an asynchronous job	Optional

The recommended starting mode is synchronous, with stable analysis identifiers and status fields so that asynchronous jobs can be added if measured latency requires it.

10.4.3 Request and response

The illustrative payloads below are not real inference output.

```
POST /internal/v1/analyze
Authorization: Bearer <INTERNAL_SERVICE_TOKEN>
X-Request-ID: <correlation-id>
{
  "analysis_id": "an_123",
  "evidence_id": "ev_123",
  "claim_id": "cl_456",
  "image": { "source_type": "internal_url",
```

```
            "source": "https://trusted-storage.example/signed-object",
            "sha256": null },
  "options": { "damage_detection": true, "image_quality": true,
               "exact_duplicate_check": true,
               "perceptual_duplicate_check": true,
               "embedding_similarity": false, "top_k": 5 }
}
```

```
Response:
{
  "analysis_id": "an_123",
  "evidence_id": "ev_123",
  "status": "completed",
  "damage": { "status": "completed",
              "detections": [ { "class_name": "dent", "confidence": 0.91,
                                "box_xyxy": [120, 85, 360, 290] } ],
              "model_name": "damage-detector", "model_version": "mock-v1" },
  "image_quality": { "status": "completed", "label": "acceptable", "issues": [] },
  "duplicates": { "exact_match": false, "exact_match_evidence_ids": [],
                  "perceptual_candidates": [] },
  "similarity": { "status": "not_requested", "matches": [] },
  "warnings": [],
  "created_at": "2026-10-21T10:00:00Z"
}
```

The image reference is a short-lived signed URL or a controlled internal reference created by NestJS. FastAPI never accepts arbitrary public URLs or filesystem paths, which would allow server-side request forgery or file access. If it fetches a URL, it allows only trusted storage hosts, blocks localhost, private, and link-local targets, limits redirects, and enforces timeouts and byte limits. The overall status is `partial` when an optional component fails but other results are valid.

10.4.4 Model adapters

The route code contains no model-specific logic. A mock detector and the real YOLO detector implement the same interface and produce the same schema, so replacing the model changes only the adapter, the model configuration, and the evaluation tests. The model is loaded once at startup, inference runs under `torch.inference_mode()` with the training preprocessing reproduced, thresholds are configuration values, and the model version is logged with each result. Readiness is reported accurately: a required missing model fails readiness, an optional component is marked unavailable, and mock results are visibly marked and disabled in production by default.

10.4.5 Integrating the trained models

The trained detector and the similarity index are delivered as a package containing the module `ethioclaim_ml.py` (classes `DamageDetector` and `SimilarityIndex`), the weights `damage_best.pt`, the gallery embeddings (374 vectors of 384 values) with file names, pinned requirements, and a README. Their outputs map onto the service schema as follows.

Service schema	Package output	Adapter action
`class_name`	`damage_type`	Rename.
`box_xyxy`	`bbox`	Rename; both use x_1, y_1, x_2, y_2 pixels.
`model_name`, `model_version`	`model_version`	Add a model name.

Service schema	Package output	Adapter action
Similar match `evidence_id`	`file`	Map the stored file name to the evidence ID.
Similar match `score` (0 to 1)	`similarity`	Clamp to the 0 to 1 range.
Similar match `method`	(implicit)	Set to `embedding`.
`image_quality`	not built	Return status `unavailable`.

The packaged gallery holds embeddings of the CarDD test images and serves only to seed a demonstration. In production, the index holds embeddings of uploaded evidence, stored with the embedding model name and version, and excludes the current evidence ID from its results. The detector was trained with Python 3.13 and Ultralytics 8.4.183, so the same library versions are pinned and loading of the weights is tested in the deployment environment. *[Update this paragraph with the actual integration status once the backend developer confirms it.]*

10.4.6 Intake limits and errors

Setting	Starting value	Reason
Maximum upload	10 MB	Avoid unbounded memory use; tune to real phone photos.
Longest dimension	6000 px	Avoid pathological decoded images.
Formats	JPEG, PNG	Small decoder surface; WebP only if tested.
Storage fetch timeout	5 to 15 s	Avoid hung downloads.
Analysis timeout	30 to 60 s	Tune after a hardware benchmark.
Similarity top-k	5 default, 20 maximum	Bound the response size.

Images are validated by content, not only by extension or MIME type. EXIF orientation is normalized and the color space is RGB. The original is preserved and a temporary copy is preprocessed. Service errors use stable codes: `INVALID_IMAGE_REFERENCE` (400), `UNAUTHORIZED_SERVICE` (401), `IMAGE_TOO_LARGE` (413), `UNSUPPORTED_IMAGE_TYPE` (415), `REQUEST_VALIDATION_ERROR` (422), `ANALYSIS_CAPACITY_EXCEEDED` (429), `IMAGE_SOURCE_UNAVAILABLE` (502), `AI_SERVICE_NOT_READY` (503), `ANALYSIS_TIMEOUT` (504), and `INTERNAL_ERROR` (500). Responses never contain stack traces, file paths, credentials, or signed URLs.

10.4.7 Request flow

1. The frontend uploads evidence to NestJS.
2. NestJS authenticates and authorizes the user, validates the file, stores the original, and records an evidence ID.
3. NestJS calls the analysis endpoint with a service token and a request ID.
4. FastAPI validates the image, hashes it, runs the available adapters, and returns structured output.
5. NestJS persists the output linked to the evidence and claim.

6. The frontend reads the result from NestJS and shows AI findings separately from the examiner's assessment.
7. The examiner uses the findings to guide the review. A similarity hit alone does not determine fraud.

10.5 Model readiness gate

Damage detection is not considered complete merely because a model package installs. The readiness check covers sample images, the output schema, inference latency, CPU and GPU requirements, false positives, and failure behavior. Current status: the held-out evaluation, output schema, and per-class error analysis are done, GPU latency is measured, and CPU requirements and failure behavior in the deployed service remain to be tested. If a component is not ready, it is exposed as unavailable instead of pretending to work.

11 Security, Privacy, and Auditability

11.1 Authentication

Passwords are hashed with a modern algorithm and never stored in plaintext. Browser sessions use secure HTTP-only cookies where feasible, or a carefully designed token strategy, over HTTPS in deployed environments. Login and policy lookup are rate-limited, accounts can be deactivated and sessions revoked, and login errors are generic.

11.2 Authorization and data access

Role and record scope are checked in NestJS for every list, detail, download, update, and retry endpoint. Uploaded images are never publicly accessible through guessable URLs; they are served from private storage after authorization, and signed URLs expire quickly. Policy verification returns minimal data, and an examiner cannot read another examiner's claim unless the team policy allows it.

11.3 File upload protection

The backend allowlists image types and verifies file signatures where feasible, sets maximum file size and maximum photo count, generates storage keys on the server, protects against path traversal and malformed uploads, keeps the sanitized original filename only as metadata, restricts download and deletion, and preserves evidence history for submitted claims.

11.4 Audit events

Audited events include login failures where appropriate, user creation and deactivation, role changes, policy creation, edits, and status changes, claim creation, submission, assignment, and status transitions, evidence upload, removal, and replacement, analysis retries, assessment submission and revision, final decisions, and claim reopening. Each event records the actor, action, entity, timestamp, and reason or metadata. The application offers no operation that edits or deletes audit events, and passwords, tokens, and unnecessary personal data are never logged.

11.5 Privacy, retention, and configuration

Only data necessary for claim handling is collected, and visibility of names, phone numbers, photographs, and linked claim history is defined per role. A retention and deletion policy must be agreed with the insurer before real deployment. For the demonstration, fictional data is used and no real personal documents or claimant photographs are uploaded without permission. Secrets live in environment configuration, not in source control. Required variables are validated at startup without printing their values, development, test, and production configurations are separate, FastAPI stays internal, and health endpoints do not expose diagnostics to unauthenticated clients. The AI service additionally avoids putting personal data into embeddings, returns only evidence IDs and technical match details (NestJS decides what the examiner may see), and never logs image bytes, authorization headers, or signed URLs.

12 Validation and User Experience

12.1 Policy, claim, and evidence validation

- Policy numbers are normalized according to the insurer's rules, and empty or invalid input is rejected. An unknown policy returns a clear not-found result without creating a policy.
- Status and effective dates are checked at verification and again at submission, which handles a policy being deactivated after the first lookup.
- A claim requires a policy identifier, incident date and time, and incident description. Invalid dates are rejected, only permitted status transitions are allowed, the vehicle must belong to the policy, and the claim number is generated by the backend.
- Evidence checks cover size, type, and claim access. Metadata is persisted before analysis, upload retries do not create unintended duplicates, upload failure is distinguished from analysis failure, and manual review stays available while analysis is pending, unavailable, or failed.

12.2 Accessibility and low-bandwidth use

Compact layouts, full-resolution images loaded only when opened, upload progress with retry, clear nontechnical error messages, keyboard-accessible and properly labelled controls, status shown by text and icons and not by color alone, and draft state preserved without storing sensitive policyholder details in insecure persistent browser storage.

13 Testing and Acceptance Criteria

13.1 Role and authorization tests

An Intake User cannot create or edit a policy through direct API requests. An Examiner cannot create users, change roles, edit policies, or open an unassigned claim. An Intake User cannot submit a claim for an ineligible policy. Direct API calls are rejected even when the interface hides the button, and inactive users cannot use protected endpoints.

13.2 Policy, claim, and evidence tests

An Admin can create a policyholder, policy, and vehicle association. A duplicate policy number for the same insurer is rejected, a valid lookup returns the permitted summary and prefills the form,

and an unknown policy leaks no unrelated records. An expired or cancelled policy is flagged, a vehicle not linked to the policy is rejected, and later policy edits do not rewrite the snapshot of a submitted claim. An Intake User can save a draft, upload images, review, and submit; the claim reference is generated by the backend; an exact duplicate is identified by SHA-256; a near-duplicate can be flagged by the perceptual hash; an analysis timeout does not delete the claim or evidence; and unauthorized users cannot download evidence by guessing an identifier.

13.3 Assessment tests

An Examiner can record severity, a manual ETB estimate, observations, and a recommendation. Invalid enumeration values are rejected by the backend, assessment submission creates an audit and timeline event, an AI similarity warning alone cannot trigger automatic rejection, and the final-decision endpoint, if included, requires the proper permission and a reason.

13.4 AI service tests

Unit tests cover hashing (identical files share a SHA-256, and pHash behavior on resized and recompressed copies), the image loader (corrupt, oversized, unsupported, and extreme-dimension images fail safely), the mock detector schema, similarity search (it excludes the current evidence ID, sorts the top k, and handles an empty index), partial failures, and a 401 response for a missing or wrong token. Integration tests cover authenticated calls from NestJS, invalid or expired image references, a second identical image retrieving the first evidence ID, vector entries preserving the embedding model and version, and AI failure leaving evidence and claim status untouched.

13.5 Evaluation metrics

Component	Metrics and checks
Damage detector	Precision, recall, and mAP per class; inspection of false positives and missed damage.
Exact duplicate	Correct matching of same-file evidence.
Perceptual hash	Precision and recall on resized, recompressed, cropped, and unrelated pairs; tuning of the Hamming threshold.
Embedding retrieval	Recall@K, Precision@K, and mAP@K on labelled retrieval pairs.
Image quality	False-retake rate and missed unusable photos across lighting conditions and phones.

Evaluation data should be split by vehicle, source, or claim where possible, because near-identical images in training and validation inflate metrics.

13.6 Definition of done

The MVP is ready for demonstration when a seeded Admin can create a demo policy, an Intake User can verify it, see the prefilled data, create a claim, upload evidence, and submit it, an Examiner can open the assigned claim, review images and available analysis, and either request information or submit a documented assessment, and the Admin can manage users and policies and inspect claim activity. All three roles must be blocked from unauthorized actions by the backend, and failures must be visible without losing the claim or evidence.

14 Team Split and Implementation Plan

Person	Ownership	Deliverables
Frontend developer	Next.js UI	Login, role dashboards, policy-first claim wizard, claims list and detail, evidence upload and gallery, examiner review and assessment screens, API integration, loading and error states.
Core backend developer	NestJS, PostgreSQL, Prisma	Authentication and RBAC, users, insurer, policyholder, policy, and vehicle records, policy lookup, claims, status and assignment, assessment APIs, audit log, DTO validation, integration tests.
Evidence and AI developer	FastAPI and AI models	Damage detector and similarity model training and evaluation, hashing, analysis contract and persistence support, retry behavior, NestJS integration, tests.

The team consists of Samuel Nigussie, Mussie Ferde, and Maraky Ayalneh. The members contribute across the frontend, core backend, and AI evidence-analysis workstreams; this report does not assign individual tasks where the exact allocation has not been confirmed.

14.1 Order of work

Phase	Build	Exit condition
1. Contracts and schema	Role names, DTOs, status enumerations, policy model, IDs, error shape	One shared contract with no conflicting field names.
2. Authentication and seed data	Login, role guard, demo users, insurer, policyholders, policies, vehicles	Each role can sign in and policy data exists.
3. Policy-first intake	Verification, prefill, draft claim, incident form	Intake can verify a seeded policy and create a draft.
4. Evidence upload	Private storage, metadata, preview, access checks	Images persist and only authorized users retrieve them.
5. Analysis baseline	SHA-256 and pHash, then model and embeddings	Exact-duplicate results work and analysis failure keeps evidence.
6. Examination	Assignment, review UI, information request, assessment	An examiner can complete a review.
7. Admin oversight	Policy and user management, assignment, audit, failure view	The Admin can operate the demo safely.
8. Hardening and demo	Role tests, error handling, responsive polish, seed reset, demo script	The flow works from login to assessment.

14.2 Integration rules

Commit an OpenAPI specification early. Decide whether authentication uses cookies or bearer tokens before building protected frontend calls. Agree the exact claim and evidence status enumerations before writing pages, and use the same casing for analysis statuses in NestJS and FastAPI. Keep FastAPI internal. Use ISO 8601 timestamps with a time zone and a decimal representation for ETB. Seed demo data with a repeatable script. Keep optional AI features optional so a model delay does not block the product.

15 Limitations, Ethics, and Future Work

15.1 Limitations

- The platform runs on fictional demo data and has no connection to a real insurer.
- The AI components were evaluated on public data (CarDD) and synthetic image edits. Their performance on Ethiopian vehicles, local photographs, and real claims is unknown, and the results do not establish fraud-detection ability.
- The detector is weak on cracks, scratches, and dents.
- Similarity thresholds are provisional and were tested on an easy, small gallery without hard negatives.
- The detector and similarity components were evaluated separately; no experiment shows that combining them improves fraud detection.
- Image quality assessment, severity estimation, repair-cost estimation, OCR, and document-consistency checks are not implemented.

15.2 Ethical use

AI output is advisory. The system never approves or rejects a claim, never labels a person as fraudulent, and presents a similarity result as a lead for review. Personal data is minimized, access is role- and record-scoped, and every consequential action is auditable.

15.3 Future work

- Detector improvements: higher input resolution, a larger model, and additional labelled training data (with duplicate screening against the test set and license checks), each compared on validation data and reported once on the test set.
- Local data: photographs representative of Ethiopian vehicles and claim conditions, and calibration of similarity and pHash thresholds on them, including hard negatives.
- Features: image-quality checks, severity labels, OCR and evidence-consistency checks, notifications, authorized live insurer lookup, multi-vehicle policies, advanced audit export, pgvector for large galleries, and a background job queue.

16 Conclusion

EthioClaim provides a policy-first claim intake and evidence review workflow with role-based access, private evidence storage, an audit trail, and an AI service that supports the human examiner. The

fine-tuned YOLO11n detector reaches a test mAP@50 of 0.701 and mAP@50–95 of 0.550, with strong results on shattered glass, flat tires, and broken lamps and weak results on cracks. DINOv2-small retrieves the correct original in 99.5% of 1,200 synthetic queries, clearly outperforming exact and perceptual hashing, especially on cropped and flipped images. These results come from public data and controlled edits and do not establish real-world fraud detection. The appropriate role of both components is to provide visual evidence and candidate matches to examiners, who make every claim decision.

A Demonstration Data and Test Scenarios

The demonstration environment may use fictional policyholders, insurance policies, vehicles, claims, and evidence images. Suggested test scenarios include active, expired, and cancelled policies; claims in different processing stages; and identical or visually similar images.

These scenarios help demonstrate policy validation, claim intake, evidence handling, exact duplicate detection, and AI-based image similarity search. Fictional records should be used for demonstrations unless appropriate authorization has been obtained to use real data.

B AI Artifacts and Evaluation Outputs

The following artifacts support the training, evaluation, and integration of the EthioClaim AI components. The paths should be updated to match the actual project directory structure.

Artifact	**Location**
Best damage detector	`ethioclaim/cardd\_yolo11n-3/weights/best.pt`
Last-epoch checkpoint	`ethioclaim/cardd\_yolo11n-3/weights/last.pt`
Training curves and confusion matrix	`ethioclaim/cardd\_yolo11n-3/`
Test evaluation outputs	`ethioclaim/cardd\_yolo11n\_test/`
Similarity results by method	`ethioclaim/similarity\_results.csv`
Threshold evaluation results	`ethioclaim/similarity\_threshold.csv`
Similarity failure examples	`ethioclaim/similarity\_failures.png`
Gallery embeddings and filenames	`ethioclaim/handoff\_ml/gallery\_embeddings.npy`
Backend integration package	`ethioclaim/handoff\_ml/`

C Limitations and Future Considerations

The current prototype supports human review by providing vehicle-damage detection and image-similarity results. These AI outputs are advisory and do not independently determine whether a claim is fraudulent or should be approved or rejected.

Further development should address deployment requirements, access control, evidence retention, data protection, and validation using representative real-world insurance claim images. “‘

References

[1] X. Wang, W. Li, and Z. Wu, "CarDD: A New Dataset for Vision-based Car Damage Detection," *IEEE Transactions on Intelligent Transportation Systems*, 2023 (arXiv:2211.00945). *[Verify the citation details before submission.]*

[2] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," 2023 (model `facebook/dinov2-small`).

[3] Ultralytics, YOLO11 documentation and software, version 8.4.183.

[4] Hugging Face dataset `shanexf/cardd-damage` (YOLO-format conversion of CarDD).